Digital Design Verification

Lab Manual # 17 – RISC-V Datapath Implementation (Loads, Stores and Jumps)

Release: 1.0

Date: 11-June-2024

NUST Chip Design Centre (NCDC), Islamabad, Pakistan

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	11/06/2024	Ali Aqdas	Complete manual	-
2.0	19/08/2025	Umer Farooq / Fawad Ahmed	Manual Update	-

Contents

Objective

The objective of this lab is to

- Extend the previously designed datapath to include loads, stores and jumps.

Tools

- Cadence Xcelium
- Xilinx Vivado

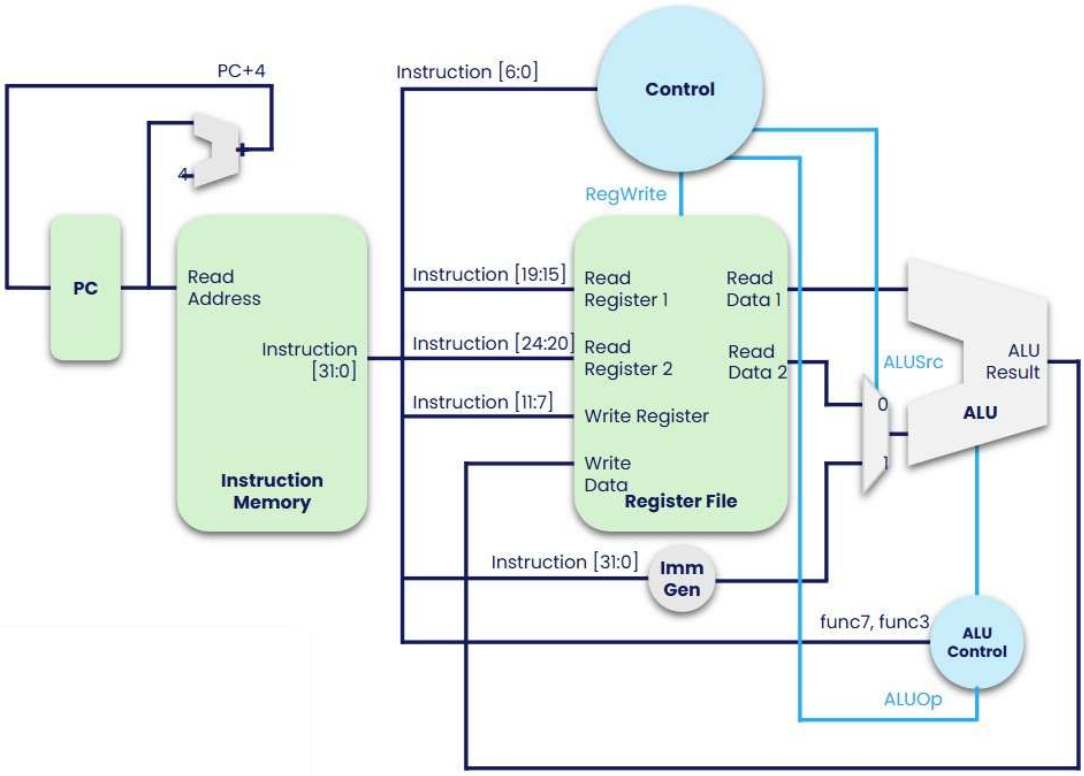
Instructions

The students are required to design and submit the updated files for evaluation of this lab. An additional file for data memory has to be added in the submission.

./<student-name>/	**./rtl/**	program_counter.sv inst_mem.sv data_mem.sv reg_file.sv imm_gen.sv alu_logic.sv alu_ctrl.sv main_ctrl.sv top.sv
	./tb/	program_counter_tb.sv inst_mem_tb.sv data_mem_tb.sv reg_file_tb.sv imm_gen_tb.sv alu_logic_tb.sv alu_ctrl_tb.sv main_ctrl_tb.sv top_tb.sv

RISC-V Datapath

The datapath designed so far is shown in the figure below.



It includes all the **R-Type** and **I-Type Instructions** (of course **excluding loads**, which cannot be implemented without a data memory).

Main Memory

Memory is also a **state element** that holds both instructions and data in one contiguous 32-bit memory space. The behavior of memory elements is a follows

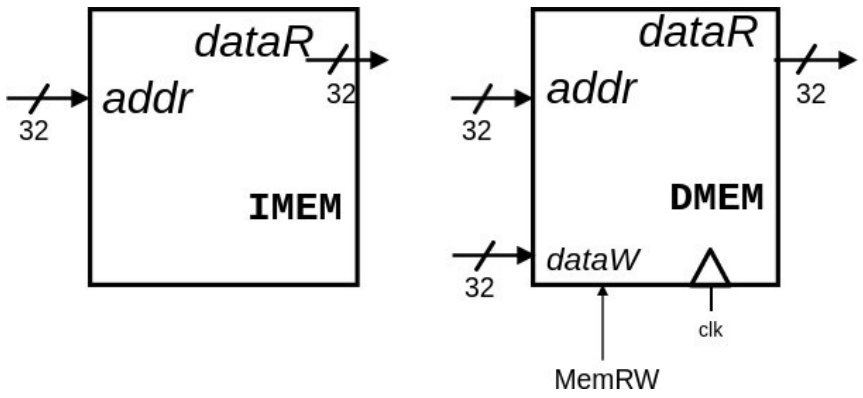
Inputs			Outputs		
Function	**Size**	**Port Name**	**Function**	**Size**	**Port Name**
Input Address	32-Bit	addr	Data Output	32-Bit	dataR
Input Data	32-Bit	dataW			
Write Enable (1=Enable,0 = Disable)	1-Bit	MemRW			

Reading a memory location does not require clock tick to trigger. However, a clock trigger causes the memory to write the data of the previous cycle

Generally, memory is partitioned into two parts, one for the instructions and one for the data, however for simplicity, we are going to create two memories.

1. **Instruction Memory:** A read only memory for fetching instructions. Since it's a read-only memory it never requires a clock trigger.

2. **Data Memory:** A memory for loading (reads) and storing (writes) of data words. Under the hood, these are just placeholders for caches (more on this later).



In this lab, you shall design the data memory and associated logic to enable load and store operations.

It may be noted that while **loads** are **I-Type** Instructions, the **stores** are **S-Type**. A brief description and list of these instructions is given in the sections below

Loads (I-Type)
The load instructions are encoded as I-Type Instructions.

A list of load instructions is given in the table below.

op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	–	I	lb rd, imm(rs1)	load byte	rd = SignExt([Address]7:0)
0000011 (3)	001	–	I	lh rd, imm(rs1)	load half	rd = SignExt([Address]15:0)
0000011 (3)	010	–	I	lw rd, imm(rs1)	load word	rd = [Address]31:0
0000011 (3)	100	–	I	lbu rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address]7:0)
0000011 (3)	101	–	I	lhu rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address]15:0)

The load instructions function in a similar way to **add** instructions, with immediate operands. Address is computed inside the ALU and propagated to the data memory to index the required data from memory array.

Stores (S-Type)
The store instructions are encoded in a special instruction type known as the S-Type (bear in mind that S does not stand for Special). The encoding of S-Type is as follows:

It is slightly different from the I-Type. The destination register operand is replaced with the immediate value while the source operand (rs2) is brought back, leading to breaking down of immediate into two fields. Breaking down ensures uniformity in the fields (rs1, rs2, rd, etc.).

A list of store instructions is given in the table below

op	funct3	funct7	Type	Instruction	Description	Operation
0100011 (35)	000	–	S	sb rs2, imm(rs1)	store byte	$[Address]_{7:0} = rs2_{7:0}$
0100011 (35)	001	–	S	sh rs2, imm(rs1)	store half	$[Address]_{15:0} = rs2_{15:0}$
0100011 (35)	010	–	S	sw rs2, imm(rs1)	store word	$[Address]_{31:0} = rs2$

Memory Alignment

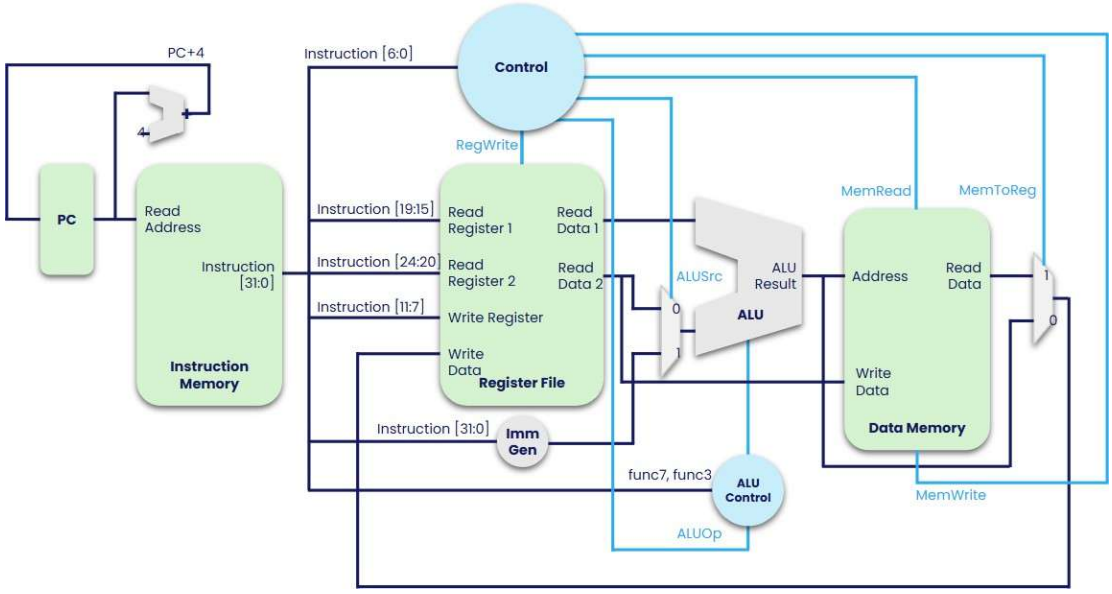
To simplify the circuitry, the data memory can only access 32-Bits at a time. Regardless of what the load/store instruction you may have **dataR** and **dataW** will always be 32-bits long. In addition, many RISC-V Memories have a further restriction that data memory can only access addresses that are multiples of 4. RISC-V often mandates that loads/stores happen on aligned addresses, that is

1. **lw/sw** happens at addresses that are multiples of 4.
2. **lh/sh** happens at addresses that are multiples of 2.
3. **lb/sb** can happen at any address.
4. **lhu (Load Halfword Unsigned):** Loads a 16-bit halfword from memory into a register and zero-extends it to 32 bits.
5. **lbu (Load Byte Unsigned):** Loads an 8-bit byte from memory into a register and zero-extends it to 32 bits.

Unaligned accesses have undefined behavior. In the case of loads, the half-words/bytes may be
selected from the 32-Bit word and Sign/Zero extended based on the instruction. In the case of stores, data smaller than word size may be manipulated to align with the right bytes, and a mask may be applied to avoid unnecessary overwrites (separate read/write bits).

Task 1

Extend the previous design to include load and store instructions in your datapath. After extending the design your datapath may look along the lines of this figure.

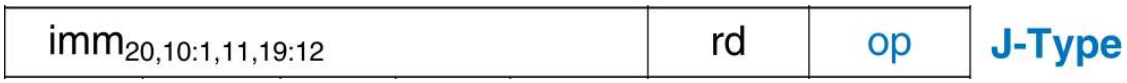


Jumps

There are two jump instructions namely

1. jalr **(I-Type)**
2. jal **(J-Type)**

The encoding of J-Type is given in the figure below



A list of jump instructions is provided in the table below

op	funct3	funct7	Type	Instruction	Description	Operation
1100111 (103)	000	–	I	`jalr rd, rs1, imm`	jump and link register	`PC = rs1 + SignExt(imm), rd = PC + 4`
1101111 (111)	–	–	J	`jal rd, label`	jump and link	`PC = JTA, rd = PC + 4`

The **jal** instruction jumps to the address specified in the immediate field, while the **jalr** uses the addresses stored in **rs1** along with immediate values to make longer jumps.

Task 2

Extend the design from previous task to include the jump instructions. After extending the datapath your design may look along the lines of the following figure of jal & jalr instructions